Assignment3:

AIM: Design and implement a symbol table for a compiler that stores information about identifiers such as name, data type, scope, and memory location. Perform operations like insertion, lookup, and deletion of entries.

C Program:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define SIZE 10

struct Symbol {
    char name[50];
    char datatype[20];
    char scope[20];
    int memory_location;
    struct Symbol* next;
};

struct Symbol* table[SIZE];

int hash(char* name) {
    int sum = 0;
    for (int i = 0; name[i] != '\0'; i++) {
        sum += name[i];
    }
    return sum % SIZE;
}
void insert(char* name, char* datatype, char* scope, int mem_loc) {
    int index = hash(name);
```

```
    struct Symbol* newSymbol = (struct Symbol*)malloc(sizeof(struct Symbol));
    strcpy(newSymbol->name, name);
    strcpy(newSymbol->datatype, datatype);
    strcpy(newSymbol->scope, scope);
    newSymbol->memory_location = mem_loc;
    newSymbol->next = NULL;

    newSymbol->next = table[index];
    table[index] = newSymbol;

    printf("Inserted successfully!\n");
}

void lookup(char* name) {
    int index = hash(name);
    struct Symbol* temp = table[index];

    while (temp != NULL) {
        if (strcmp(temp->name, name) == 0) {
            printf("\nSymbol Found:\n");
            printf("Name: %s\n", temp->name);
            printf("Datatype: %s\n", temp->datatype);
            printf("Scope: %s\n", temp->scope);
            printf("Memory Location: %d\n", temp->memory_location);
            return;
        }
        temp = temp->next;
    }

    printf("Symbol not found!\n");
}

void deleteSymbol(char* name) {
```

```
    int index = hash(name);
    struct Symbol* temp = table[index];
    struct Symbol* prev = NULL;

    while (temp != NULL) {
        if (strcmp(temp->name, name) == 0) {

            if (prev == NULL)
                table[index] = temp->next;
            else
                prev->next = temp->next;

            free(temp);
            printf("Deleted successfully!\n");
            return;
        }

        prev = temp;
        temp = temp->next;
    }

    printf("Symbol not found!\n");
}

void display() {
    printf("\n--- Symbol Table ---\n");
    for (int i = 0; i < SIZE; i++) {
        struct Symbol* temp = table[i];
        while (temp != NULL) {
            printf("Name: %s | Type: %s | Scope: %s | Mem: %d\n",
                   temp->name, temp->datatype, temp->scope,
temp->memory_location);
            temp = temp->next;
```

```
        }
    }
}

int main() {
    int choice;
    char name[50], datatype[20], scope[20];
    int mem_loc;
    for (int i = 0; i < SIZE; i++) {
        table[i] = NULL;
    }

    while (1) {
        printf("\n1. Insert\n2. Lookup\n3. Delete\n4. Display\n5. Exit\n");
        printf("Enter choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                printf("Enter name: ");
                scanf("%s", name);
                printf("Enter datatype: ");
                scanf("%s", datatype);
                printf("Enter scope: ");
                scanf("%s", scope);
                printf("Enter memory location: ");
                scanf("%d", &mem_loc);
                insert(name, datatype, scope, mem_loc);
                break;

            case 2:
                printf("Enter name to search: ");
                scanf("%s", name);
```

```
            lookup(name);
            break;

        case 3:
            printf("Enter name to delete: ");
            scanf("%s", name);
            deleteSymbol(name);
            break;

        case 4:
            display();
            break;

        case 5:
            exit(0);

        default:
            printf("Invalid choice!\n");
      }
   }

   return 0;
}
```